Mini-Project 2: Quadruped Locomotion

MICRO-507 Legged Robots - Groupe 23

Zaynab Hajroun
EPFL
SCIPER: 346235

Yasmine Tligui
EPFL
SCIPER: 341215

Thalia Riachy
EPFL
SCIPER: 346308

***Abstract*—Our work explores and compares Central Pattern Generators and Deep Reinforcement Learning for dynamic locomotion of a quadruped robot. The CPG approach uses Hopf oscillators for rhythmic movement. In this method, adding a cartesian PD controller improved the foot trajectory accuracy compared to joint-space control alone.**

For the DRL, the proximal policy optimization (PPO) algorithm was trained. Results indicate that the trained trotting policy achieved a stable average forward velocity on flat terrain with minimal lateral drift but the system also demonstrates robustness by adapting to inclined terrains after fine tuning and domain randomization.

I. Introduction

Developing dynamic locomotion for legged robots remains a major challenge in the field, with critical applications in search and rescue, healthcare, and autonomous transport. These robots are required to navigate diverse terrains while maintaining energy efficiency and robust stability. While traditional control methods often rely on predefined gaits that lack flexibility, recent advancements have shifted toward Deep Reinforcement Learning to optimize adaptive and robust behaviors. Another effective approach involves biologically inspired networks known as Central Pattern Generators, which provide a natural framework for rhythmic movement without requiring high-level planning.

This project explores the implementation of these two distinct strategies CPGs and DRL to enable a quadruped robot to achieve stable locomotion. We first model the robot's kinematics and implement a CPG based on coupled Hopf oscillators, using both joint and cartesian space PD controllers to track desired foot trajectories. We then detail the design of a Markov Decision Process for DRL-based control. By comparing the their performance, this report evaluates the two methods.

II. Methodology

Part 1- Central Pattern Generators

A. Quadruped Modeling

The robot simulation is using a quadruped model, each leg is actuated by three motors corresponding to the hip, thigh, and calf joints, resulting in a total of 12 degrees of freedom.

1) Kinematics: Calculations are performed relative to the leg frame.

The relationship between joint angles q and foot position p is governed by the forward kinematics function $f(\cdot)$, such that $p = f(q)$. Differentiating this relationship yields the foot velocity v:

$$v = J(q) \cdot \dot{q} \tag{1}$$

where $J(q) \in \mathbb{R}^{3\times3}$ is the single-leg Jacobian. This Jacobian is also essential for mapping forces F, at the foot, back to joint torques τ:

$$\tau = J^T(q)F \tag{2}$$

2) Control: To track the desired motion, we implement both Joint space and Cartesian space PD controllers.

Joint PD Control: The torque for leg i is computed based on the error between desired $(q_d, \dot{q}_d)$ and current $(q, \dot{q})$ joint states:

$$\tau_{joint} = K_{p,joint}(q_d - q) + K_{d,joint}(\dot{q}_d - \dot{q}) \tag{3}$$

where $K_{p,joint}$ and $K_{d,joint}$ are the proportional and derivative gain vectors, respectively .

Cartesian PD Control: To improve tracking performance, particularly for the stance phase, we compute torques required to track desired foot positions (p_d) and velocities (v_d) in the leg frame using the Jacobian transpose:

$$\tau_{Cartesian} = J^T(q)[K_{p,Cart}(p_d - p) + K_{d,Cart}(v_d - v)] \tag{4}$$

where $K_{p,Cart}$ and $K_{d,Cart}$ are diagonal matrices of proportional and derivative gains.

B. Central Pattern Generators

We implement locomotion controllers using CPG, based on coupled Cartesian space Hopf oscillators, which doesn't require higher-level planning.

The time evolution of the state of the oscillator for leg i is defined by the following differential equations:

$$\dot{r}_i = a(\mu - r_i^2)r_i \tag{5}$$

$$\dot{\theta}_i = \omega_i + \sum_{j=0}^{3} r_j w_{ij} \sin(\theta_j - \theta_i - \phi_{ij}) \tag{6}$$

where r_i is the amplitude of the leg and θ_i its phase, $\sqrt{\mu}$ the desired amplitude, a is a convergence constant, and ω_i is the natural frequency. The coupling between oscillators is defined by the weight w_{ij} and the desired phase offset ϕ_{ij}, which enforces the coordination required for specific gaits (trot, walk, pace, bound).
The specific coordination required for different locomotion patterns is achieved by setting a unique phase coupling matrix ϕ_{ij}, which dictates the desired phase offsets between the four oscillators to produce gaits such as trot, walk, or pace.

The natural frequency ω_i is split into swing (ω_{swing}) and stance (ω_{stance}) components to match different speeds and phases:

- **Swing Phase:** $0 \leq \theta_i \leq \pi$ (foot in air).

- **Stance Phase:** $\pi < \theta_i \leq 2\pi$ (foot in contact).

The CPG states (r, θ) are mapped to desired Cartesian foot positions (x_{foot}, z_{foot}) in the leg frame. This mapping generates the foot trajectory:

$$x_{foot} = -d_{step} r_i \cos(\theta_i) \tag{7}$$

$$z_{foot} = \begin{cases} -h + g_c \sin(\theta_i) & \text{if } \sin(\theta_i) > 0 \\ -h + g_p \sin(\theta_i) & \text{otherwise} \end{cases} \tag{8}$$

where d_{step} is the step length, h is the nominal robot height, g_c is the ground clearance during swing, and g_p is the ground penetration during stance. These desired Cartesian positions are then tracked using the PD controllers described in Section II-A.

Part 2: Deep Reinforcement Learning (DRL)

C. Observation and Action Space Design

1) Observation Space: Our observation space includes joint angles and velocities, body orientation (roll, pitch, yaw), base linear and angular velocities, body height z, ground reaction forces, foot contact states, and CPG amplitudes and phases: 50 dimensions total.

We chose Euler angles over quaternions for direct interpretability: roll, pitch, and yaw have clear physical meanings used directly in our reward function, whereas quaternions are purely mathematical with no intuitive interpretation. This also reduces dimensionality (3 vs 4). We include only body height z rather than full position (x, y, z) because absolute location is irrelevant for our locomotion tasks, while height indicates falls and terrain changes and was clearly relevant during training. The principle of observation space is not to put everything we observe, but to include only features that improve stability, policy learning and decision-making, while minimizing dimensionality and noise. We include only normal forces and contact booleans from `GetContactInfo()`, excluding redundant contact counts and unreliable self-collision detections. CPG states include only amplitudes and phases, not derivatives, as derivatives add noise sensitivity without additional information. We exclude previous actions because CPG states already encode temporal locomotion patterns and joint states reflect action effects, avoiding overfitting to memorized action sequences.

All measurements are hardware-feasible via joint encoders, IMUs, and contact sensors. Joint encoders are highly accurate, while IMUs exhibit moderate noise and drift in orientation and angular velocity measurements. Contact force sensors are the noisiest due to sensor limitations and motor current estimation errors. We applied proportional Gaussian noise (1% of each observation's range) during training to simulate sensor imperfections.

Observation normalization is critical because measurements span vastly different scales (angles $\sim$1 rad, velocities $\sim$10 rad/s, forces $\sim$1000 N). Without normalization, large values dominate gradients and the network ignores smaller features. Normalizing to consistent ranges ensures equal feature contribution and faster convergence.

2) Action Space: Actions are bounded to $[-1, 1]$ following the normalization principle: different action dimensions have vastly different physical scales.

We chose CPG-RL over direct joint or Cartesian PD control because because it guides the learning process by starting from known stable gaits, rather than learning from scratch. Also, CPG allows us to train directly on specific gaits of interest by setting the appropriate phase coupling matrix, reduces dimensionality from 12 to 8 actions for faster learning, generates smooth coordinated motion, and has proven hardware transferability. We added coupling between oscillators to enforce inter-leg coordination. Without coupling, each leg oscillates independently in simulation, leading to uncoordinated, inefficient gaits and worse results.

D. Robustness and Sim-to-Real Transfer

Our approach should transfer reasonably well to real hardware: CPG-based control generates smooth trajectories suitable for actuators, all observations are hardware-feasible (encoders, IMU, contact sensors), and we applied domain randomization during training. However, sim-to-real transfer faces challenges including unmodeled dynamics (motor delays, backlash, structural compliance), higher sensor noise than simulated, terrain variations beyond training distribution, and contact model discrepancies between PyBullet's rigid-body assumptions and real compliant contacts. Sim-to-sim transfer to other simulators may require retraining due to different contact solvers and integration schemes.

E. Deep Reinforcement Learning Policy

We used Proximal Policy Optimization (PPO) rather than SAC because PPO is an on-policy algorithm better suited for continuous control tasks with smooth, stable learning dynamics. PPO uses a clipped surrogate objective to prevent large policy updates, ensuring training stability-critical for physically simulated robots where catastrophic policy changes can cause failures. SAC, while sample-efficient, is off-policy and can be less stable in high-dimensional action spaces.

We retained the default hyperparameters provided in the codebase. These hyperparameters proved well-suited for our tasks. The clip range of 0.2 limits policy updates to prevent destabilizing changes, appropriate for our physically grounded robot. The learning rate of 1×10^{-4} is standard for PPO and provides stable convergence. The combination of 2048 rollout samples, batch size 128, and 10 epochs yields $(2048/128) \times 10 = 160$ gradient updates per policy rollout, providing a good balance between sample efficiency and computational cost. We did not require extensive exploration tuning because CPG-RL starts from a known stable gait (TROT), already providing a strong behavioral prior.

For fine-tuning on slopes, we adjusted only two hyperparameters: the learning rate was lowered to 1×10^{-5} (10× smaller) because the policy already learned stable locomotion. A smaller learning rate allows gentle adaptation to slopes without destabilizing the existing walking policy, and the number of iterations was reduced from 1,000,000 to 500,000, because fine-tuning converges faster than training from scratch.

F. Reward Function

Designing an effective reward function is critical for learning stable, efficient locomotion. Our reward functions balance multiple objectives: forward progress, energy efficiency, stability, and trajectory tracking. We structure rewards to penalize undesirable behaviors while encouraging the desired locomotion pattern.

1) Forward Locomotion: For flat terrain, our primary goal is to achieve steady forward walking at a target velocity range while maintaining stability and minimizing lateral drift. The reward function is:

$$r = \max\Big(r_{\text{vel}} - r_{\text{drift}} - r_{\text{orient}} - r_{\text{energy}}, 0\Big) \tag{9}$$

where each term addresses a specific aspect of locomotion:

- r_{vel}: Velocity tracking reward, encouraging forward motion within a target range (0.2–1.0 m/s). This prevents the trivial solution of standing still while discouraging unrealistic high speeds that exploit simulator dynamics.
- r_{drift}: Lateral drift penalty on both lateral position y and lateral velocity v_y to maintain straight-line motion. Initially, we penalized only position y, with lower weights, but the robot corrected deviations only after significant drift had occurred. We increased the position penalty weight and added the velocity_y term to provide early corrective feedback, enabling the robot to react to lateral motion before large deviations accumulate and preventing oscillatory corrections.
- r_{orient}: Orientation penalty on roll, pitch, and yaw deviations from upright posture, ensuring the robot maintains balance. We penalize yaw deviations twice as heavily as roll and pitch because maintaining heading direction is critical for straight-line locomotion. Unlike roll and pitch, which benefit from gravitational restoring forces (a tilted robot naturally experiences corrective torques that pull it back toward upright), yaw has no such natural correction mechanism. On yaw, once the robot begins rotating, it continues in that direction indefinitely unless actively corrected. This makes yaw errors accumulate over time. Therefore, we assign a higher penalty weight to yaw to explicitly enforce heading stability, while allowing the natural dynamics to assist with roll and pitch control. Initial training runs with equal penalty weights confirmed this issue: the robot learned stable walking but frequently rotated around the vertical axis (yaw drift), failing to maintain a straight path.
- r_{energy}: Energy consumption penalty computed from motor torques and velocities, encouraging efficient gaits that minimize actuator effort.

The reward is clipped to be non-negative to avoid negative reinforcement that can destabilize training.

2) Slopes Locomotion: Slopes introduce additional challenges: the robot must adapt body posture to handle inclines and declines while maintaining forward progress. Rather than training from scratch, we fine-tune the converged flat-terrain policy on slopes terrain. Starting from a robot that already mastered stable walking allows faster learning of the policy only needs to acquire terrain-specific adaptations (pitch management, adaptive step length). We extended the flat terrain reward with terrain-specific terms:

$$\begin{aligned} r = \max\Big(&r_{\text{vel}} + r_{\text{progress}} - r_{\text{drift}} \\ &- r_{\text{stability}} + r_{\text{orient}} - r_{\text{backward}} - r_{\text{energy}}, 0\Big) \end{aligned} \tag{10}$$

The additional and modified terms are:

- r_{progress}: Forward progress reward based on Δx (positive displacement). This is critical for slopes because instantaneous velocity may drop during steep ascents, but we still want to reward forward displacement. This term ensures the robot commits to climbing rather than stalling.
- r_{drift}: Lateral drift penalty (position_y) with significantly reduced weight compared to flat terrain. Since the policy already learned strict lateral control during flat terrain training, we retain this penalty as a maintenance term to prevent catastrophic forgetting of straight-line locomotion, but reduce its weight to allow the policy to prioritize now the learning slope-specific skills (adaptive stepping, pitch control) without over-constraining lateral motion.
- r_{backward}: Binary penalty applied when moving backward ($v_x < 0$) to explicitly discourage retreat, especially during difficult ascents where the robot might slip backward.
- r_{orient}: Base orientation penalty on absolute roll, pitch, and yaw angles. Initially, we attempted to remove the pitch penalty on slopes, reasoning that pitch must naturally adapt to terrain inclination. However, this led to increased instability, particularly at slope transitions (flat-to-incline, flat-to-descent and descent-to-flat) where pitch changes abruptly. We reintroduced the pitch penalty to suppress excessive pitch deviations while still allowing moderate terrain-adaptive pitch through the relatively low coefficient. The perfect way would be to penalize the difference between robot pitch and terrain slope angle, but this approach requires real-time terrain inclinometer sensors unavailable on the physical robot and would not generalize across varying slope configurations.
- $r_{\text{stability}}$: Base angular velocity penalty on roll, pitch, and yaw rates (ω_{roll}, ω_{pitch}, ω_{yaw}) to prevent rapid rotational oscillations. Rapid angular velocities indicate dynamic instability and are particularly common when transitioning between terrain segments (flat-to-slope, ascent-to-descent). We use both stability and orientation penalties comparing to the forward locomotion task because they address complementary failure modes: orientation penalties prevent static deviations (slowly accumulating tilts in roll/yaw and free pitch adaptation), while stability penalties prevent dynamic oscillations (rapid rotational motions across all axes), resulting in smoother, more robust locomotion.

The key insight is that slopes require rewarding displacement directly (not just velocity) and penalizing instability more heavily. The combination of these terms encourages adaptive locomotion that adjusts to terrain variations while maintaining forward progress and stability.

G. Environmental Details and Domain Randomization

To improve policy robustness and facilitate sim-to-real transfer, we applied domain randomization across all training phases. For both flat terrain and slopes, we added proportional Gaussian noise (1% of each observation's range) to simulate sensor imperfections.

For slopes training specifically, we introduced additional randomization beyond flat terrain. We narrowed the ground friction coefficient range from [0.5, 1.0] to [0.5, 0.9], reducing the maximum friction by 0.1 to better reflect realistic slippery conditions on inclined surfaces such as wet grass or loose gravel. This reduction is modest enough to remain physically plausible while forcing the policy to learn robust foot placement strategies that work across varying grip levels, without making the task impossibly difficult. We also randomized slope angles between 0.1 and 0.25 radians at each episode reset, ensuring the policy generalizes across different incline steepnesses rather than overfitting to a single slope configuration.

By training under varied conditions, the learned policy is expected to transfer more successfully to real hardware where friction, terrain geometry, and sensor characteristics inevitably differ from simulation.

H. PD Controller and Control Space Selection

For the CPG-based control strategy, we implemented a Proportional-Derivative (PD) controller to generate joint torques from desired positions obtained through inverse kinematics. The controller follows the standard control law:

$$\tau = K_p(q_{des} - q) + K_d(\dot{q}_{des} - \dot{q}),$$

where K_p and K_d are the proportional and derivative gains, q_{des} are the desired joint angles from IK, and q, $\dot{q}$ are the measured joint positions and velocities.

We chose joint-space control over Cartesian control for several reasons. First, it provides direct correspondence with the robot's actuators, avoiding additional Jacobian inverse computations required in Cartesian space. Second, it ensures better numerical stability, especially near kinematic singularities where the Jacobian becomes ill-conditioned. The PD gains used ($K_p = [100, 100, 100]$ Nm/rad and $K_d = [2.0, 2.0, 2.0]$ Nms/rad per joint) were tuned empirically to achieve accurate trajectory tracking while maintaining system stability.

III. Results

A. CPG Performance

The CPG controller generates stable rhythmic gaits. The implementation of Cartesian PD control improves the performance by providing smoother joint trajectories and more precise foot position tracking compared to joint-space control alone.

The following results compare the standard Joint PD control against the enhanced Cartesian PD control.

- **Phase Stability:** The phase θ follows a consistent periodic sawtooth profile, as shown in Figure 1, ensuring a stable rhythmic gait for all four legs (FL, FR, RL, RR).

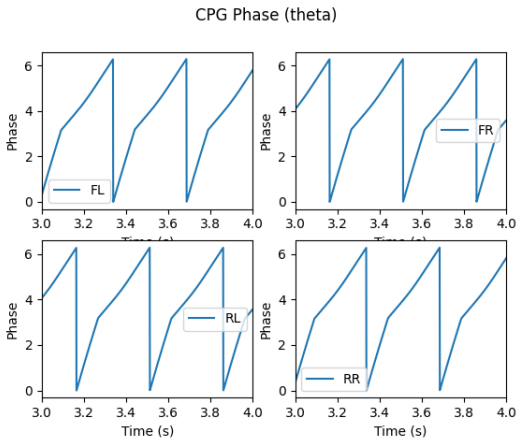


Fig. 1: CPG phase evolution for the trot gait.

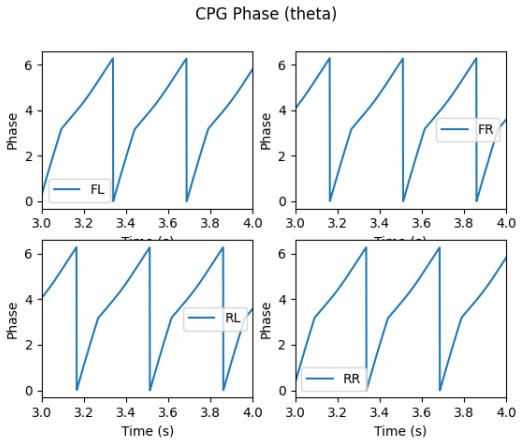


Fig. 2: CPG Phase (θ) - Without Cartesian PD

- **Phase Velocity:** As shown in Figure 3, the phase velocity ($\dot{\theta}$) alternates between swing and stance frequencies to maintain the timing required for the trot gait.

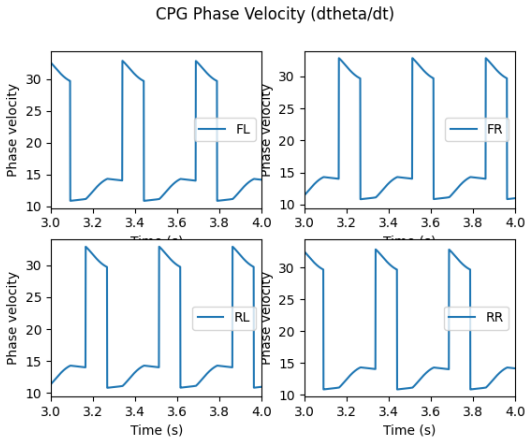


Fig. 3: Phase Velocity (With Cartesian PD)

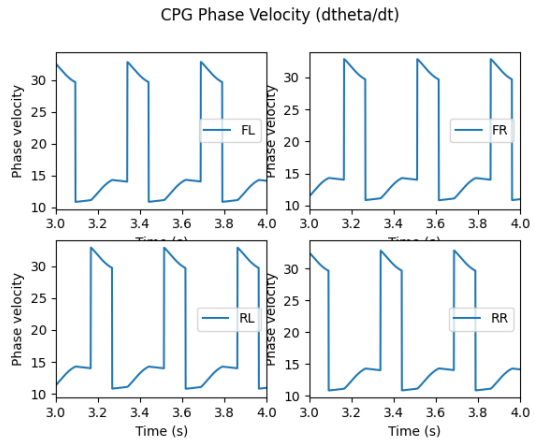


Fig. 4: Phase Velocity (Only joint PD).

- **Amplitude Consistency:** The amplitude r remains constant at its desired value regardless of the feedback controller used, confirming that the internal generator states are decoupled from the low-level tracking errors (Figure 5 and Figure 6).

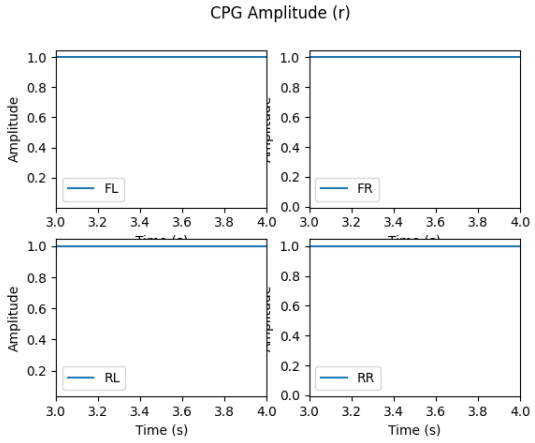


Fig. 5: CPG Amplitude (r) - Without Cartesian PD

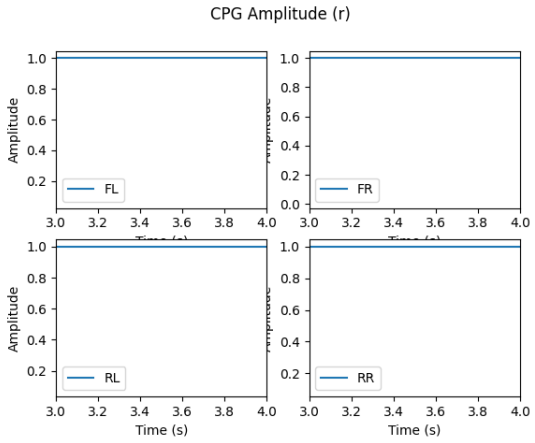


Fig. 6: CPG Amplitude (r) - With Cartesian PD

We can also see that the choice of controller impacts the robot's ability to follow the trajectories generated by the CPG.

- **Joint Angle Tracking:** Under Joint PD control alone, notable tracking errors and noise are observed, particularly in the hip and calf angles (Figure 7).

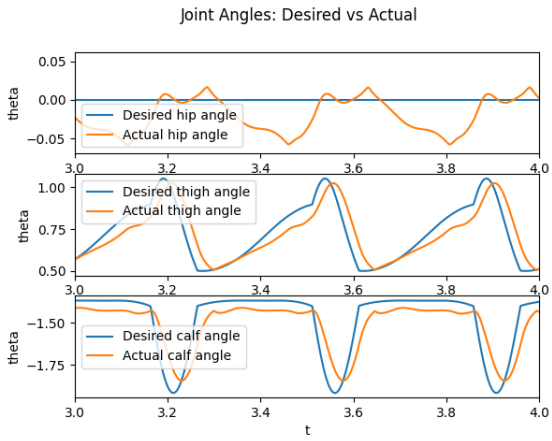


Fig. 7: Joint Angle Tracking (Joint PD Only). Note the tracking error and noise in the hip/calf angles.

- **Cartesian Improvement:** With the addition of Cartesian PD control, the actual joint angles follow the desired trajectories much more smoothly (Figure 8).

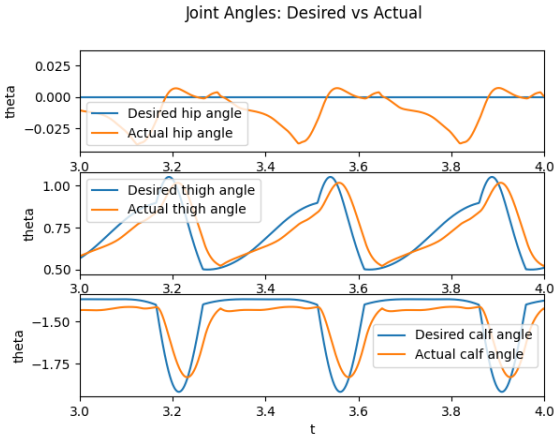


Fig. 8: Joint Angle Tracking (With Cartesian PD).

- **Foot Position Tracking:** Figure 9 highlights that the Cartesian PD controller allows the foot to follow the desired path much more closely, especially in the Z-axis (height). This improvement is critical for ensuring proper ground clearance and stable foot placement.

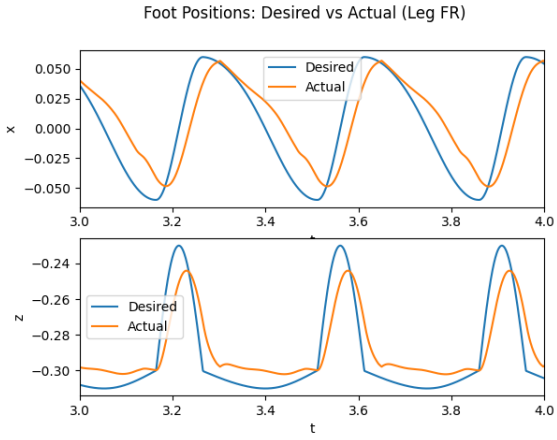


Fig. 9: Foot Position Tracking (With Cartesian PD). The actual trajectory follows the desired path closely in the Z-axis.

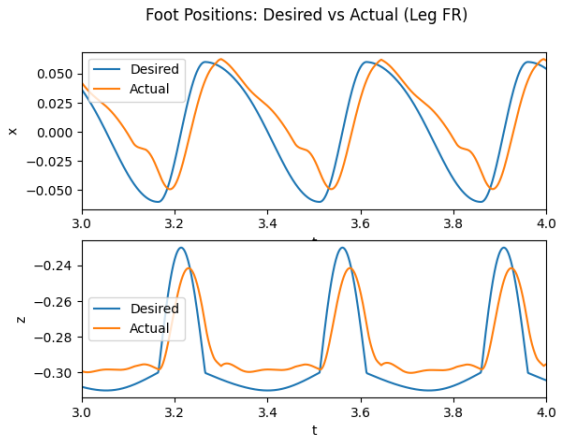
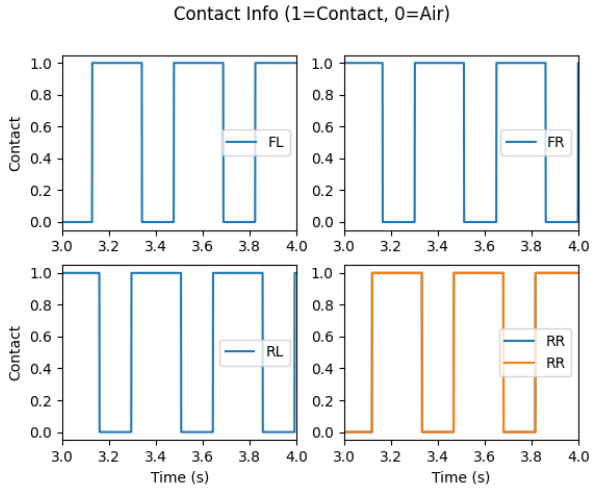


Fig. 10: Foot Position Tracking (Only joint PD).

The stability of the generated gait is further confirmed by the contact detection plots in Figure 11a. It demonstrates a regular and alternating duty cycle between leg pairs, which is characteristic of a stable trot.



(a) Contact Detection - With Cartesian PD

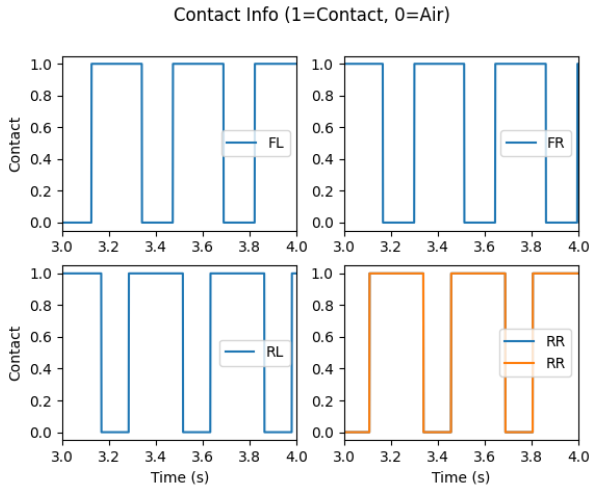


Fig. 12: Contact Detection - Without Cartesian PD

The contact stability is improved with Cartesian PD, showing more consistent contact intervals and reduced oscillation during the stance phase compared to joint-only tracking.

B. DRL Performance

1) Forward Walking Analysis: We trained our reward using 2 types of gaits: Walk and Trot.

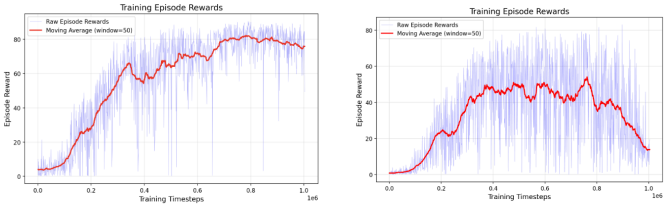


Fig. 13: Trotting (left) and walking (right) gait episode reward results for forward motion.

The learning curve exhibits three distinct phases. The initial exploration phase (0-200k timesteps) shows the agent gradually discovering basic locomotion strategies, with rewards climbing from near zero. Around 200k timesteps, the policy achieves a breakthrough, rapidly reaching the 60-80 reward range and maintaining high performance through 800k timesteps. However, the final phase (800k-1000k timesteps) shows performance degradation, likely due to over-optimization in an already-converged region where PPO clipping may introduce instability.

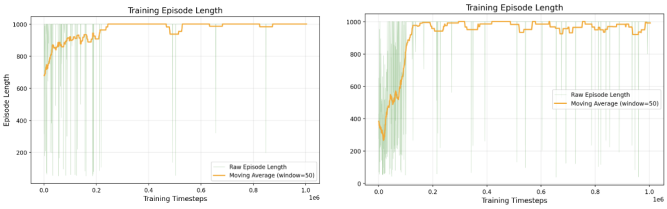


Fig. 14: Trotting (left) and walking (right) gait episode length results for forward motion.

Despite a good convergence of the episode length, the walking policy showed extreme reward oscillations throughout the entire training period compared to trotting who also converges and has a good reward evolution. Unlike trotting, which maintains two diagonal feet in contact with the ground at all times, walking requires single-leg support phases. This fundamental instability explains why the walking gait achieved lower performance despite requiring similar training duration, and motivated our choice of trotting for subsequent terrain adaptation experiments.

Also, visual analysis of the trained walking policy reveals critical stability issues during execution. Video recordings show that foot contacts are highly delicate and inconsistent, with the robot frequently exhibiting hesitation phases where legs remain suspended in air longer than necessary, as if the policy is uncertain about the optimal placement timing. This indecisiveness in leg placement contrasts sharply with the trotting gait, where diagonal leg pairs move in synchronized, confident motions.

To select the optimal policy for evaluation, we saved model checkpoints every 30,000 timesteps during training and evaluated their qualitative performance through visual inspection. For the walking gait, the best checkpoint was identified at 540,000 timesteps, while the trotting gait achieved optimal performance at 870,000 timesteps.

These selections correspond to regions of maximum sustained reward in the training curves, where the policies demonstrated both high average performance and relative stability before degradation phases.

Average and Maximum Velocity: During evaluation on flat terrain, the trained trotting policy achieved an average forward velocity of 1.14 m/s with relatively stable performance throughout the 1000-timestep episode. The velocity profile shows the robot quickly accelerates from rest to its cruising speed within the first 100 timesteps, then maintains consistent forward locomotion. Peak velocities reached approximately 1.5 m/s during certain phases, demonstrating the policy's capacity for dynamic movement. The lateral (v_y) and vertical (v_z) velocities remain close to zero with periodic oscillations characteristic of the trotting gait cycle. The vertical velocity oscillates between approximately ±0.3 m/s due to the natural bouncing motion inherent to trotting: as diagonal leg pairs alternately push off and land, the robot's center of mass undergoes cyclical vertical displacement. In contrast, the walking gait achieved a lower average velocity of 0.93 m/s with significantly higher variance, exhibiting frequent velocity drops and less stable forward progression, confirming the coordination difficulties observed during training.

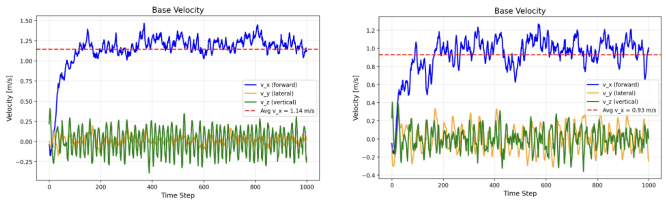


Fig. 15: Trotting (left) and walking (right) gait base velocity results for forward motion.

Position Progression: The base position plot demonstrates linear forward progress, with the y-position remains near zero (within ±0.2m at the end of the path), confirming minimal lateral drift. The z-position maintains stable height around 0.3m with small periodic oscillations characteristic of quadrupedal locomotion, indicating consistent ground clearance and balance throughout execution.

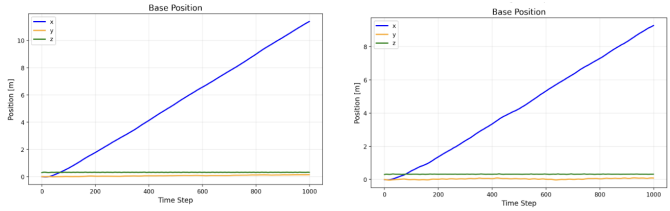


Fig. 16: Trotting (left) and walking (right) gait base position results for forward motion.

2) Slope Walking Analysis: Given the superior performance of the trotting gait on flat terrain, we employed transfer learning by fine-tuning this policy for slope traversal with an additional 500,000 training steps using a slope-specific reward function. The training curves reveal that the policy successfully adapted to slopes, reaching episode rewards around 75 and consistently achieving maximum episode lengths of 1000 timesteps after approximately 400,000 fine-tuning steps.

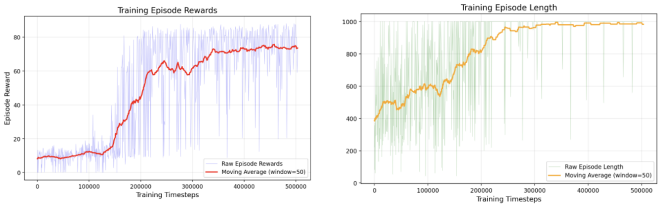


Fig. 17: Trotting gait results for slope motion.

Velocity and Position Analysis on Slopes: The velocity profile shows adaptation to terrain changes. Forward velocity initially drops to 0.3-0.5 m/s as the robot transitions from flat ground to ascending slope. As the policy adapts during ascent (timesteps 150-400), v_x increases to 0.8-1.0 m/s. During descent (timesteps 400-600), gravity assists motion, pushing velocity to 1.2-1.5 m/s. The sharp drop around timestep 600 corresponds to the slope-to-flat transition where the robot must readjust dynamics. The average speed is 0.9m, a little bit less than during the forward motion.

Stability issues concentrate at terrain transitions, from flat surface to slope and slope to flate surface. Lateral velocity (v_y) shows ±0.3 m/s oscillations, most pronounced during the slope-to-flat transition (timesteps 400-600). Vertical velocity (v_z) exhibits extreme spikes (±0.7 m/s) at the same transition zone.

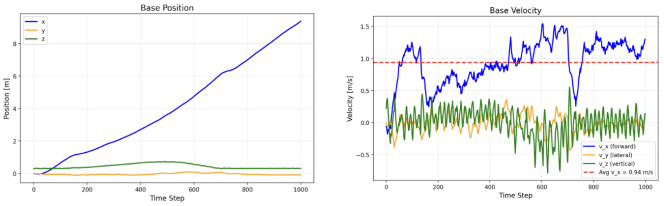


Fig. 18: Trotting gait base position and velocity results for slope motion.

Position data confirms y-drift reaches ±0.5m (2.5× worse than flat terrain), with largest deviations at terrain interfaces, and the z-position exhibits larger oscillations, suggesting reduced balance stability on inclined surfaces. This reveals the policy handles constant slopes adequately but struggles with abrupt inclination changes at flat-to-slope and slope-to-flat boundaries.

Potential Improvements: The primary issue is lateral drift (±0.5m) at terrain transitions. Potential solutions include: (1) Adding more precise drift penalties and pitch compensation terms to the reward function, (2) Incorporating center of mass (CoM) velocity tracking to detect and correct lateral instability proactively, as CoM displacement is a key predictor of balance loss in legged systems, and (3) Implementing smoother velocity commands during terrain transitions through a hierarchical controller that explicitly manages flat-to-slope interfaces.

IV. Discussion

This project compared two locomotion controllers: a CPG-based controller with PD tracking, and a PPO-based DRL controller. Both achieved stable trotting in simulation. They differ in interpretability, tuning effort, and robustness.

A. CPG vs. DRL

CPG (explicit control): The CPG produces a clean and repeatable trot. The phase signals stay periodic and consistent across legs (Fig. 1). The amplitude stays at its target value (Fig. 5, Fig. 6). This makes the behavior easy to understand and debug.

DRL (learned control): PPO learns a trotting policy that reaches high reward and full episode lengths on flat ground (Fig. 13, Fig. 14). It also adapts to slopes after fine-tuning (Fig. 17). The learned controller is harder to interpret, but it can exploit dynamics that are not explicitly modeled.

B. Effect of Cartesian PD in the CPG pipeline

Adding Cartesian PD clearly improved tracking. With joint PD only, the desired joint trajectories are followed with more noise and visible tracking error (Fig. 7). With Cartesian PD, joint tracking becomes smoother and closer to the reference (Fig. 8). The same improvement is visible in task space: the foot trajectory matches the desired path more closely, especially in the vertical direction, which is critical for ground clearance (Fig. 9).

This improved tracking also stabilizes contacts. With Cartesian PD, the contact signals show a clean alternation between diagonal leg pairs, consistent with a stable trot duty cycle (Fig. 11a). In short, Cartesian feedback makes the CPG gait more reliable because it closes the loop on the variables that matter for locomotion (foot placement and contact timing).

C. DRL performance and generalization

On flat terrain, the trained trot reaches high forward speed with small lateral drift. This is confirmed by the base velocity and position plots (Fig. 15, Fig. 16). Walking was less stable than trotting: rewards oscillate strongly even when episode length converges (Fig. 13, Fig. 14). This matches what we observed in videos: walking requires single-leg support phases, which are harder to stabilize.

On slopes, the policy learned to handle steady ascent and descent. However, the plots show that the main failures happen at terrain transitions (flat-to-slope and slope-to-flat). At these transitions, we see larger v_y oscillations and spikes in v_z, and increased y drift compared to flat ground (Fig. 18). This suggests the policy learned the steady-state behavior on inclines, but did not fully learn a smooth transition strategy.

D. Robustness and sim-to-real

CPG controller: It should transfer more easily because it is structured, smooth, and easy to constrain. Gains and trajectory limits can be tuned directly to respect actuator limits. The main sim-to-real risk is contact modeling. Real contacts are compliant and noisy. Tracking performance may degrade if the ground is softer or friction varies.

DRL controller: Domain randomization and observation noise improve robustness. Still, sim-to-real remains challenging. Unmodeled delays, backlash, and sensor noise can shift the dynamics enough to break a learned policy. The slope results also show that transitions are a weak point. Real environments have many transitions (edges, small steps, uneven patches). This would likely be the first failure mode on hardware.

E. Potential improvements

- **CPG:** add higher-level feedback loops (e.g., body roll/pitch stabilization) and contact-based reflexes to reject disturbances and improve robustness during terrain changes.
- **DRL:** train explicitly on transitions (curriculum with frequent flat-to-slope changes), increase randomization (friction, mass, motor strength), and add reward terms that penalize lateral drift and large vertical velocity spikes specifically at transitions.
- **Hybrid:** keep the CPG structure but let DRL modulate a small set of parameters (frequency, amplitude, step length, body height). This keeps learning stable and improves interpretability.

V. Conclusion

This project compared a CPG-based locomotion controller and a PPO-based DRL controller for quadruped walking in simulation.

The CPG approach generated stable and repeatable trotting gaits. Adding Cartesian PD control significantly improved joint and foot trajectory tracking, resulting in smoother motion and more consistent ground contacts. The CPG framework proved easy to tune and highly interpretable, making it well suited as a reliable baseline.

The DRL controller successfully learned fast and stable trotting on flat terrain, reaching an average forward speed of about $1.14\,\mathrm{m/s}$ with low lateral drift. Walking gaits were less stable due to single-leg support phases. After fine-tuning, the trotting policy adapted to slopes, but stability degraded at terrain transitions, where increased lateral drift and vertical velocity spikes were observed.

Overall, CPG control offers robustness and interpretability, while DRL provides higher performance and adaptability once trained. Combining both approaches through CPG-RL proved effective. Future work should focus on improving robustness at terrain transitions and strengthening sim-to-real transfer through additional feedback and domain randomization.

VI. Generative AI Declaration

1) We used AI for debugging purposes but also as an auto completion tool while coding (for example to generate plots more easily). We rephrased many parts of this report using AI for more fluidity because the message was way clearer and we believe it is one of the strongest advantages of generative AI. However, the dynamics and the pysics of the model needed to be understood well in order to understand how to complete the code, we used AI to understand obscur parts of the modeling explained in the assignment but not to complete the TODOs.
2) AI was very usefull for plotting. Prompt : "Generate a plot that contains 4 subplots of a parameter as a function of time and with a caption for the 4 plots" Then using autocompletion to generate similar plots for the other variables.
3) AI was less useful to determine and finetune the reward functions. Because the task required back and forth with the results obtained in order to find the most adequate reward function that led to the best result. Bad prompt:

”The robot is falling forward how can I improve the reward function”

4) Yes, using generative AI definitely made the project go more smoothly. It sped up tasks like research of information or optimizing our code, which freed us up to spend more time on the tougher challenges, like really understanding the system’s dynamics and figuring out what our results meant.
5) It helped us debugging simple errors as forgotten commas for example or having more information about what’s happening, which helped us point out the source of the problem and solve it. AI is very helpful to accelerate tasks and point out specific points that we were unable to see.

VII. ANNEXE

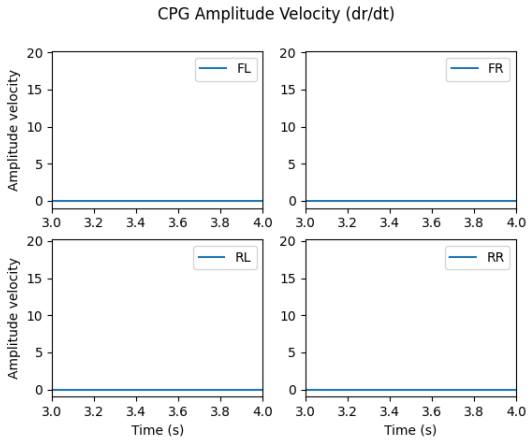


Fig. 19: Amplitude Velocity ($\dot{r}$) - Without Cartesian PD

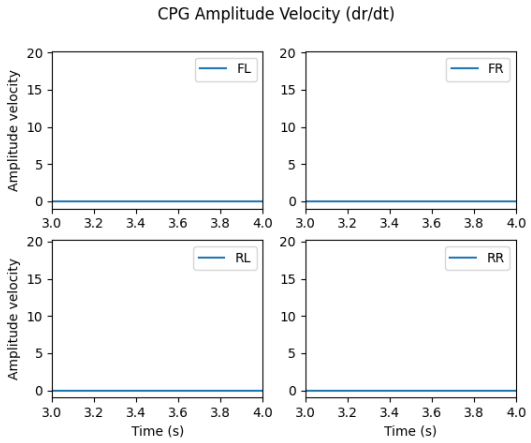


Fig. 20: Amplitude Velocity ($\dot{r}$) - With Cartesian PD